

```

/* =====
/* Function in JavaScript
/* =====
//? In JavaScript, a function is a block of reusable code that performs a specific task or set of tasks.

// 3 students at a same time wants to find the sum of two numbers

// 1st student
// var a = 5,
//   b = 10;
// var sum1 = a + b;
// console.log(sum1);

// // 2nd student
// var a = 15,
//   b = 15;
// var sum2 = a + b;
// console.log(sum2);

// // 3rd student
// var a = 55,
//   b = 15;
// var sum3 = a + b;
// console.log(sum3);

// lets make a reusable code
// function sum(a, b) {
//   return a + b;
// }

// console.log(sum(5, 5));
// console.log(sum(15, 50));
// console.log(sum(25, 750));

/* =====
/* Function Declaration:
/* =====

// function functionName() {
//   code to be excuted
//   return Result;
// }

// function greet() {
//   console.log("Hello Guys, Welcome to JS Workshop ");
// }

/* =====
/* Function Invocation:
/* =====

//?After declaring a function, you can invoke or call it by using its name followed by parentheses.
//? If the function has parameters, provide values (arguments) for those parameters inside the pare

//? How to call a function
// greet();

/* =====
/* Unparametrized Function:

```

```
/* =====
```

//? A function parameter is a variable that is listed as a part of a function declaration.

```
// Lets take an example
// function sum() {
//   var a = 10;
//   var b = 20;
//   var result = parseInt(a) + parseInt(b);
//   console.log(result);
// }
// sum();
// Main problem in this code it's not a reusable code
// Every time we get same value
```

```
/* =====
```

```
/* Parametrized Function:
```

```
/* =====
```

```
// Lets take an same example
```

```
// function sum(a, b) {
//   var result = a + b;
//   console.log(result);
// }
// sum(10, 20);
// sum(12, 25);
// sum(14, 30);
```

```
/* =====
```

```
/* Parametrized Function:
```

```
/* =====
```

//? A function argument is a value that you provide when you call a function.

//? Let's say we want to greet students with one same line

```
// function greet(name) {
//   console.log("Hello " + name + ", Welcome to JS Workshop");
// }
```

```
// greet("ramesh");
// greet("Mahesh");
```

```
/* =====
```

```
/* Function expressions
```

```
/* =====
```

//? A function expression is a way to define a function as part of an expression.

```
// var result = function sum(a, b) {
//   console.log(a + b);
// };
// result(5, 10);
```

```
/* =====
```

```
/* Anonymous Function
```

```
/* =====
```

//? An anonymous function is a function without a name.

```
// var result = function (a, b) {
//   console.log(a + b);
// };
// result(5, 10);
```

```
/* =====
/* Return Keyword
/* =====
```

```
// function sum(a, b) {
//   console.log(a + b);
//   return a + b;
// }
// var result = sum(5, 10);
// console.log(result);
// console.log(sum(5, 10));
```

```
/* =====
/* IIFE - immediately invoked function expression
/* =====
```

//? An IIFE, or Immediately Invoked Function Expression, is a JavaScript function that is defined and invoked immediately.

```
// Syntax
// var varName = (function () {
//   code to be executed
// })();
```

```
// Example
// var result = (function (a, b) {
//   // return a + b;
//   console.log(a + b);
// })(5, 10);
```

```
/* =====
/* Default Parameter
/* =====
```

// ?Default Parameter is allow the initialization of function parameters with default values.

```
// function sum(a, b) {
//   return a + b;
// }
// console.log(sum(5));
// O/P - Nan
```

```
// function sum(a, b = 10) {
//   return a + b;
// }
// console.log(sum(5));
```

```
/* =====
/* Arrow Function
/* =====
```

// ?Arrow function provide a concise syntax for writing function expressions in JavaScript.

```
// var result = function sum(a, b) {
```

```
// return a + b;  
// };  
// console.log(result(5, 10));  
  
// var result = (a, b) => {  
//   return a + b;  
// };  
// console.log(result(5, 10));
```